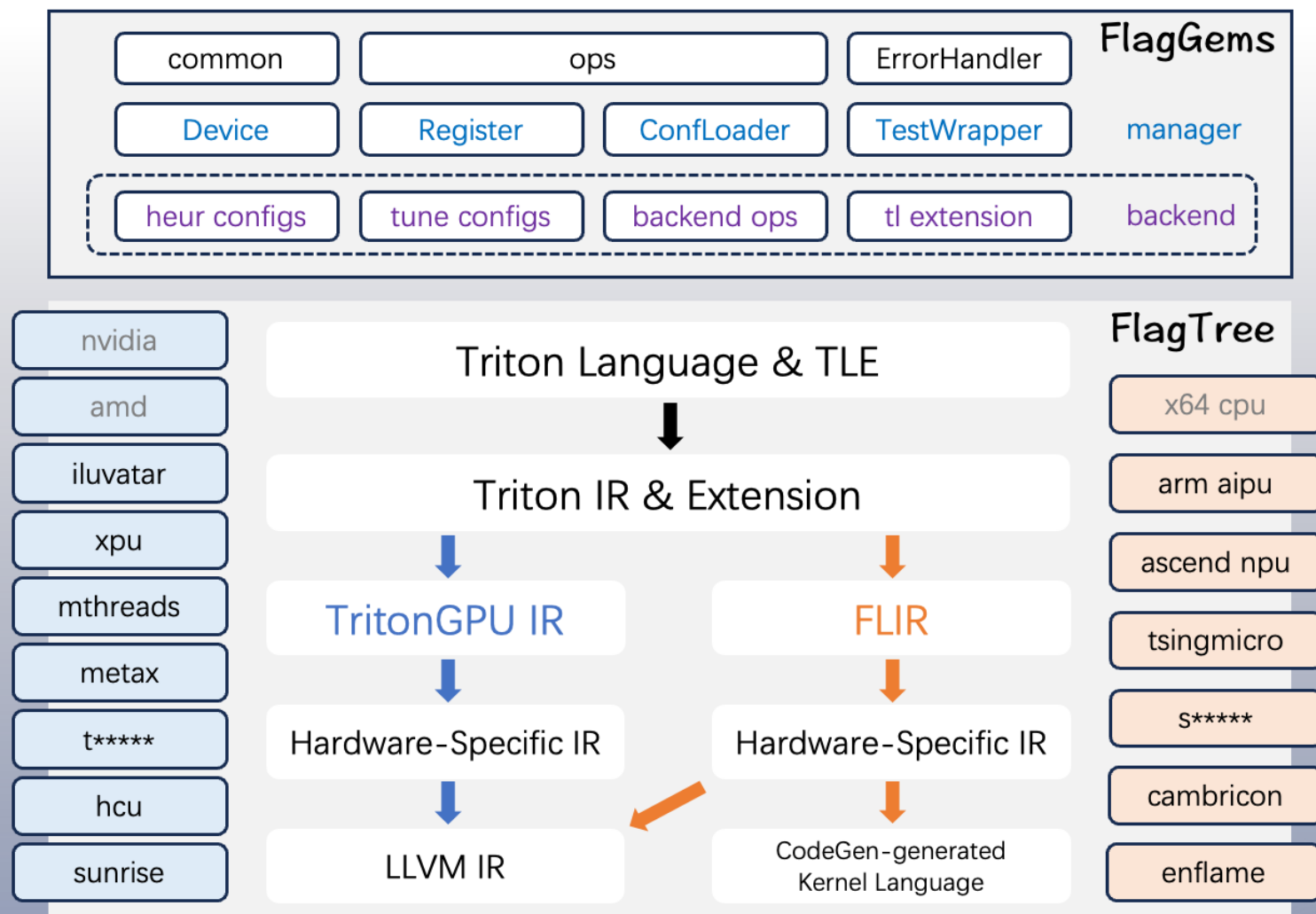


基于 FlagTree 生态的 Profiler&Debugger 初探

周炽金

水木羽林 研究员 / 华东师范大学 助理教授

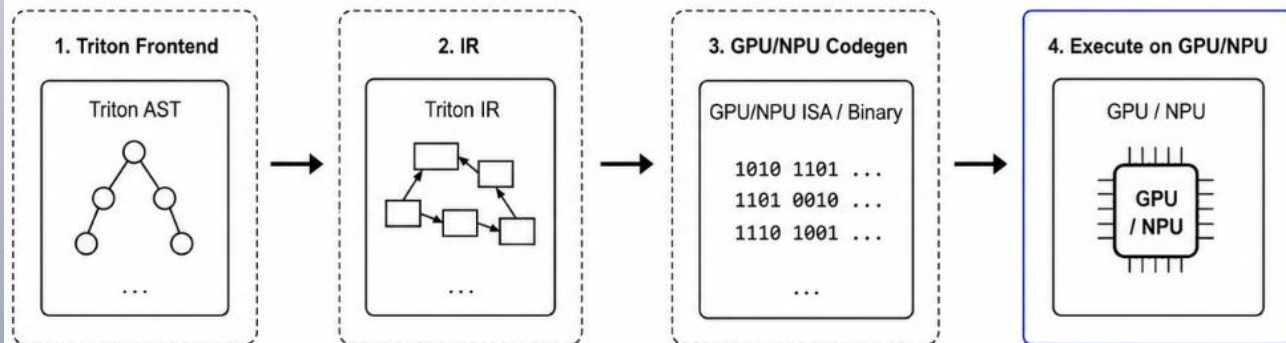


```
import triton
import triton.language as tl

@triton.jit
def add_kernel(...):
    # ...
    pid = tl.program_id(0)
    # ...
    x = tl.load(...)
    y = tl.load(...)
    tl.store(..., x + y)
    # ...
```



JIT Compilation & IR Lowering



常规 Python 调试、host 侧 profiling 无法获得细粒度的设备侧运行时信息:

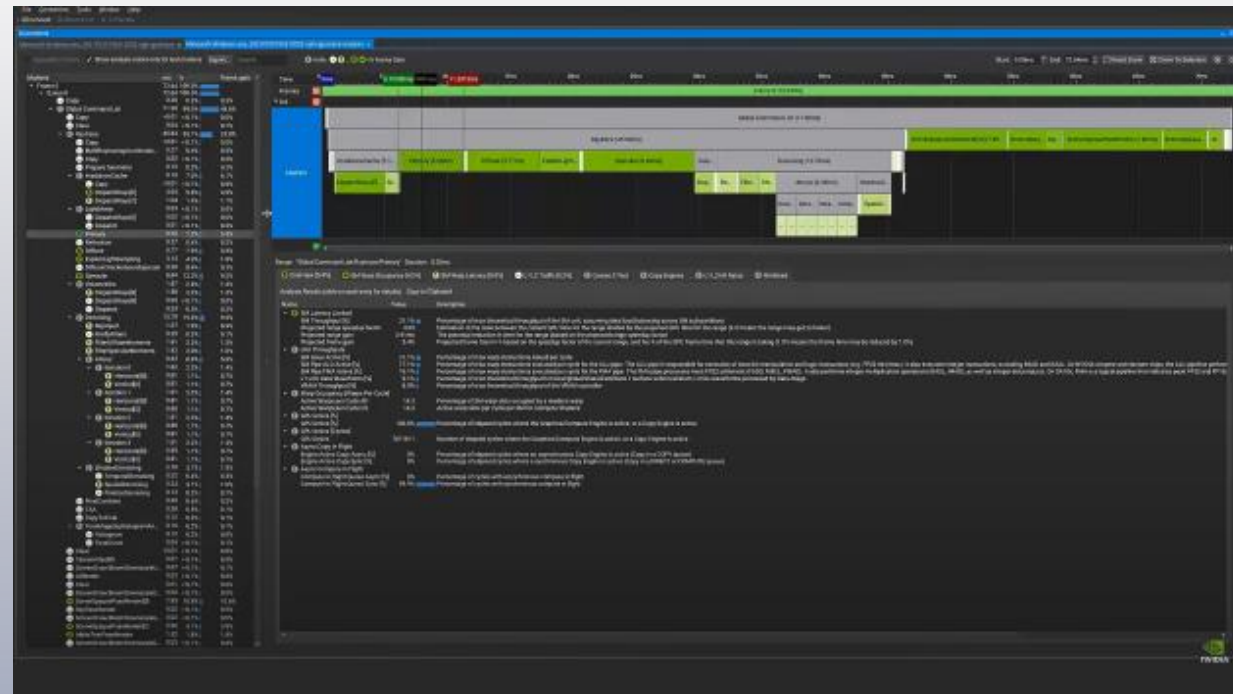
- kernel 总耗时内部由哪些操作构成? 内部操作的耗时情况是怎么样的?
- 中间计算过程中在哪一步出现了数值异常?

Profiling

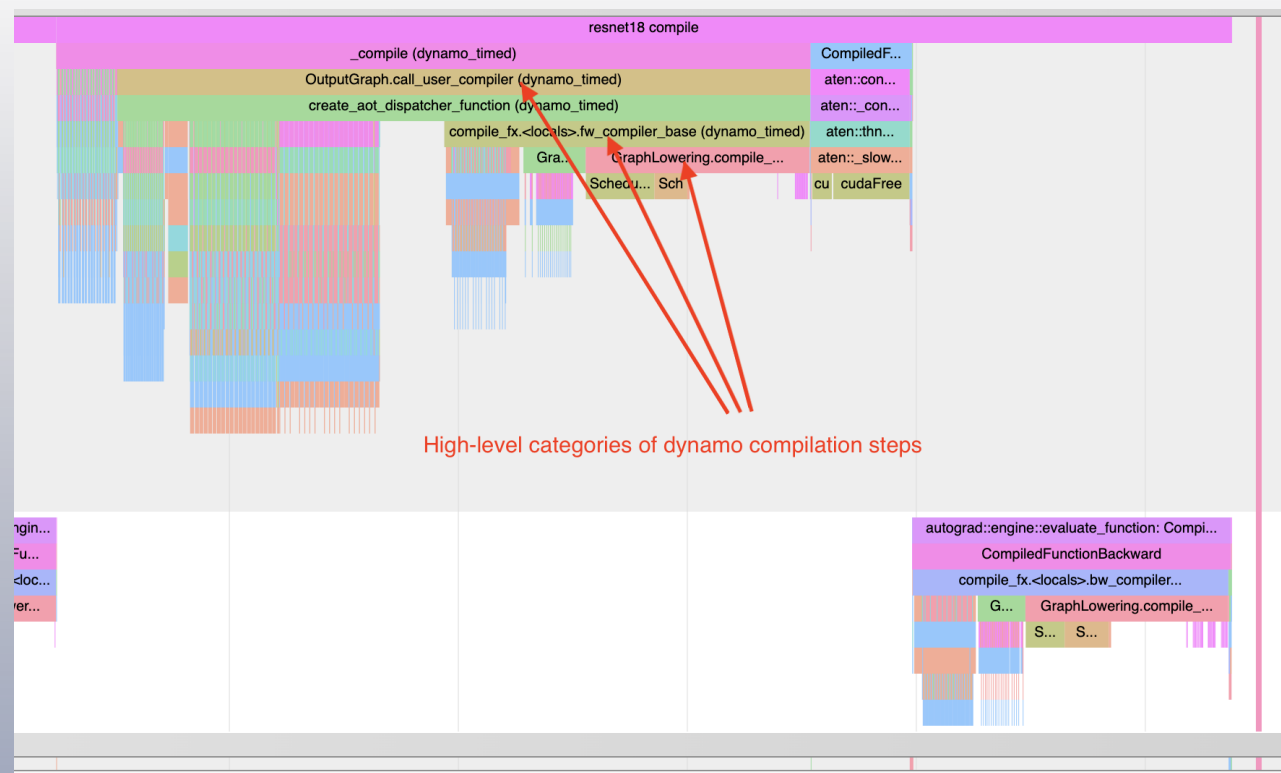
Debugging



Nvidia Nsight、CANN msprof/aclprof、Moore Perf、mcProfiler 等提供 timeline、task、硬件计数器、pipeline 和带宽指标。



PyTorch Profiler覆盖框架op、CPU调用、kernel launch; Proton面向Triton提供profiling和region记录能力。



需求

统一的工具

观测kernel内部的耗时细节

关联哪条Triton语句
对应哪些设备事件

观测哪条Triton语句
出现了数值异常

能力维度	厂商提供的 Profiler	PyTorch Profiler	Triton Proton
Python/框架 调用上下文		√	√
Triton kernel launch 信息		√	√
厂商task与硬 件计数器	√	√	
支持多个后 端芯片			
算子内部数 值地址信息			

设备端语义

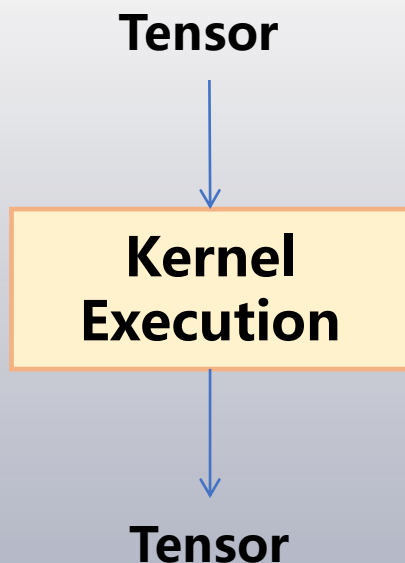
GPU / GPGPU Kernel	NPU Kernel / Task
Grid / CTA	Block
SM	AIC / AIV
Warp / Lane	Cube / Vector / Scalar / MTE
Instruction / Pipeline	GM / L1 / L0 / UB

...

算子开发语义

Python Scope
|
Triton Kernel
|
Triton Statement
|
TTIR Operation
|
Tensor / Pointer / Load / Store / Dot

一条Triton语句经过编译后形成多条设备指令；多个op经过编译器融合与重排，在设备上形成重叠执行。厂商profiler能够提供指令级信息，开发者仍无法判断瓶颈。



异常发生在kernel内部，但Kernel执行是黑盒

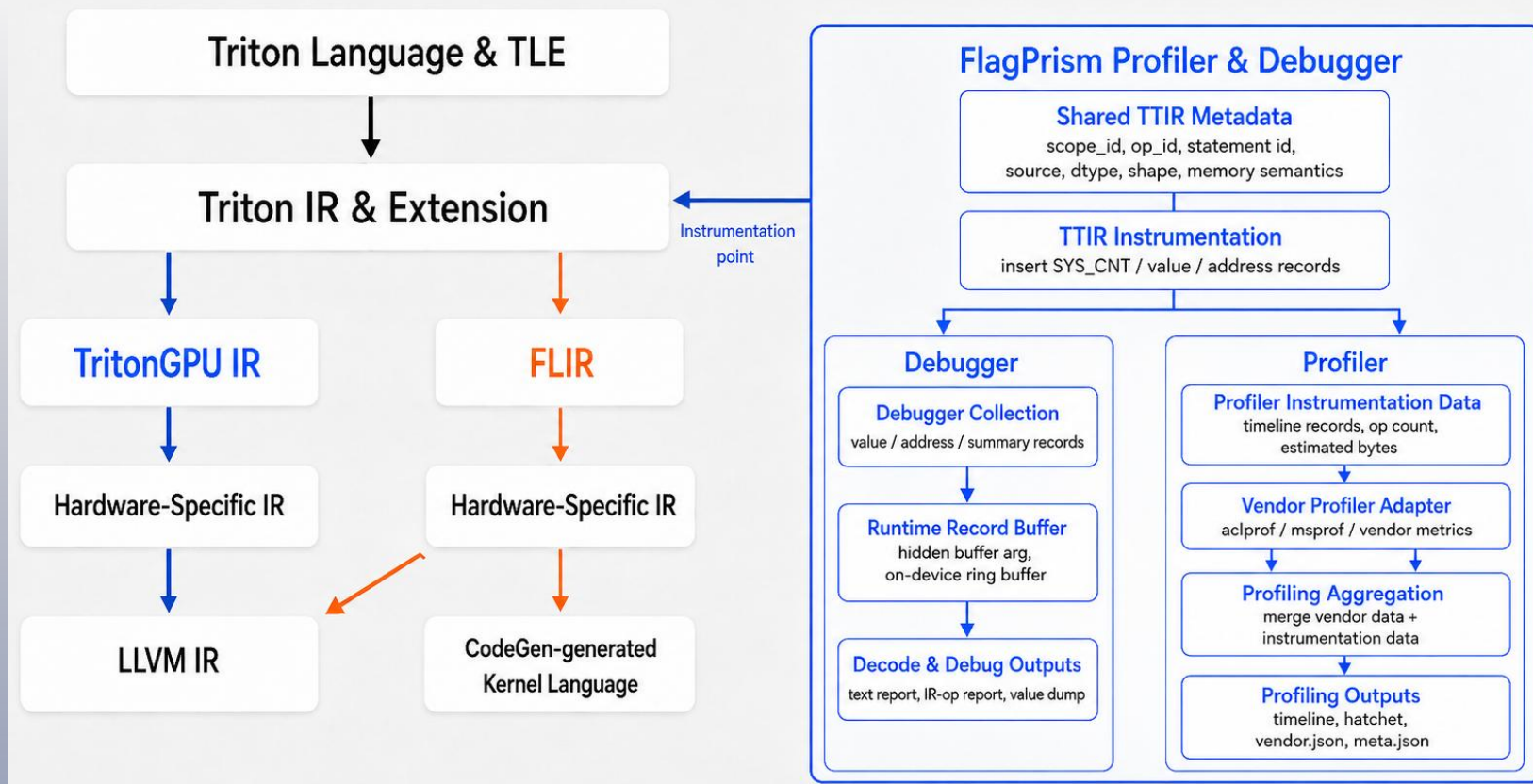
开发者想知道：某一步中间结果是否出现NaN或Inf，某次load或store的地址是否越界，mask是否按预期生效？

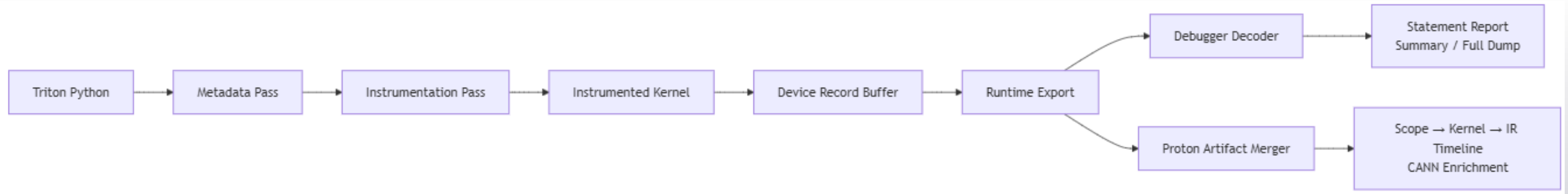


每条 Triton 语句的 Value 和 Address 信息

维度	Debugger	Profiler
分析目标	正确性、数值与访存诊断	执行结构与性能归因
观测粒度	statement / IR in kernel	statement / IR in kernel
采集范围	显式标记的 IR region	显式标记的 IR region
数据内容	summary、value、address	timestamp、duration、count、throughput、vendor metrics
主要产物	结构化报告与存value的 NPY 文件	Timeline、Hatchet、基本信息

两者共享插桩、元数据





观察哪个值？记录哪些东西？如何在设备端记录？
插入指令是否会被优化？如何映射回原语句？

新增观测 op	表达的意图	lower后的主要操作
record_summary_bundle	采集数值摘要	cmp/select/reduce
capture_memory_address	采集访存地址	ptr_to_int/reduce
record_full_value_ref	保存完整值	payload 写入
record_timeline	采集 op 周期	读 SYS_CNT 并计算差值

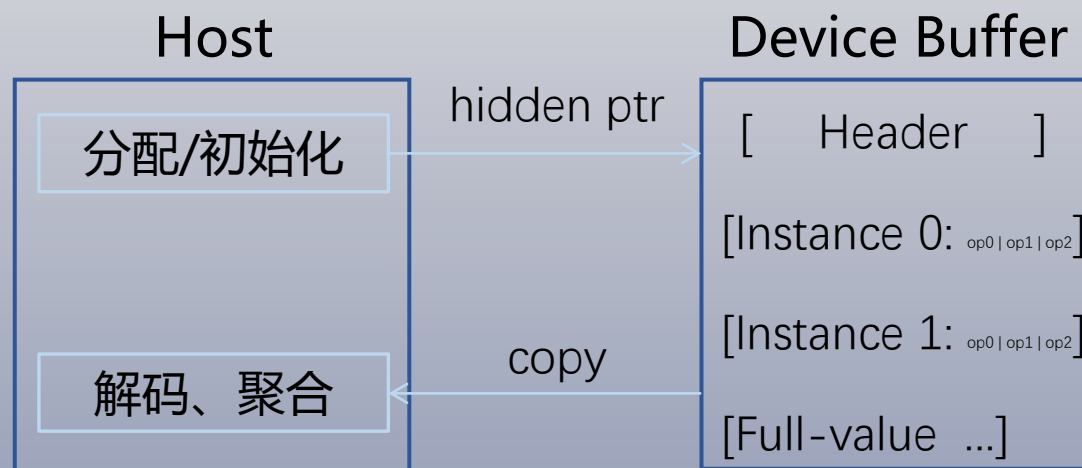
原始 IR → 选择目标 → 分配 op_id → 插入观测 op → 生成 record plan
观测 op → 取得隐藏 Buffer 参数 → 展开统计计算 → 写入确定的 slot

```
%y = arith.divf %x, %scale

flagtree_debug.record_summary_bundle %y {
  op_id = 17,
  collectors = ["nan_count", "inf_count", "mean_finite", ...]
}
```

为什么在设备端写buffer?

- Host launch hook 看不到寄存器/片上存储中的中间值，也无法得知 mask 生效后每个 lane 的实际地址。
- 每个 program instance 会并发产生记录，需要低开销、无冲突、可解码的数据通道。
- 所有观测点在 kernel 执行期间只写设备 Buffer，kernel 结束后再统一导出，从而避免多次 Device-to-Host 同步。



```
import triton.language as tl
import flagtree.debugger as debugger

debugger.configure(
    output_dir=Path("/tmp/flagtree_debugger_example"),
    record_capacity=4096,
    export_raw_records=False,
)
debugger.activate(level=1, addr_level=1)

... (运行算子)

debugger.deactivate()
```

- `level=1`：结果 summary； `level=2`：完整 tensor value
- `addr_level=1`：访存地址摘要； `addr_level=2`：完整地址序列
- `record_capacity`：record 数量上限；报告中的 overflow 表示采集不完整

```
@triton.jit
def debug_abs_kernel(x_ptr, y_ptr, n: tl.constexpr,
                    BLOCK_SIZE: tl.constexpr):
    offsets = tl.program_id(0) * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n

    tl.debug_collect_start(level=1, addr_level=1)
    x = tl.load(x_ptr + offsets, mask=mask, other=0.0)
    y = tl.abs(x)
    z = y + 1.0
    tl.store(y_ptr + offsets, z, mask=mask)
    tl.debug_collect_end()
```

start/end marker 决定动态采集范围


```
test_debug_abs_kernel_aiv_20260629_193456_497_run1.txt  
test_debug_abs_kernel_aiv_20260629_193456_497_run1.json  
test_debug_abs_kernel_aiv_20260629_193456_497_run1_op_log.txt  
test_debug_abs_kernel_aiv_20260629_193456_497_run1_op_log.json
```

← Python 语句级别的文本报告

← IR op 级别的文本报告

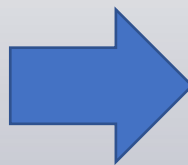
► 语句级别的文本报告

```
Triton Statement Records
source_loc: loc("x"("third_party/Debugger/examples/abs_level1.py":28:16))
statement_id: 28004
statement: x = tl.load(x_ptr + offsets, mask=mask, other=0.0)
[result x]:
  instances: [0]
  summary:
    element_count: [16 (U64)]
    nan_count      : [0 (U64)]
    inf_count      : [0 (U64)]
    zero_count     : [1 (U64)]
    mean           : [-0.5 (F32)]
    min            : [-8 (F32)]
    max            : [7 (F32)]
    l2_norm        : [18.5472 (F32)]
  address_summary(load from):
    status          : [complete]
    first_addr      : [0x12c041200000]
    last_addr       : [0x12c04120003c]
    min_addr        : [0x12c041200000]
    max_addr        : [0x12c04120003c]
    active_lane_count : [16]
    address_span_bytes: [64]
  <operand mask>:
    instances: [0]
    summary:
      element_count: [16 (U64)]
  <operand other>:
    constant_value: dense<0.000000e+00> : tensor<16xf32>
```

```
source_loc: loc("y"("third_party/Debugger/examples/abs_level1.py":29:15))
statement_id: 29004
statement: y = tl.abs(x)
[result y]:
  instances: [0]
  summary:
    element_count: [16 (U64)]
    nan_count      : [0 (U64)]
    inf_count      : [0 (U64)]
    zero_count     : [1 (U64)]
    mean           : [4 (F32)]
    min            : [0 (F32)]
    max            : [8 (F32)]
    l2_norm        : [18.5472 (F32)]
  <operand x>: [result x]
```

```
debugger.activate(level=2, addr_level=21)

tl.debug_collect_start(level=2, addr_level=2)
...
tl.debug_collect_end()
```



新增输出文件

```
*_artifacts/
  tensor_index.json
  op<id>_inst<id>_rec<id>_value.npy
  op<id>_inst<id>_rec<id>_memory_address.npy
```

- NPY 文件保留指定 op/instance 的完整 value 与 address
- 主报告只保存 artifact 引用, 避免在文本中展开大规模数据
- `tensor_index.json` 提供 record、operation、instance 到文件的索引

```
import flagtree.profiler as profiler
```

```
session_id = profiler.start(name="profile_name", context="shadow")
```

```
... (运行算子)
```

```
profiler.finalize()
```



```
def _tiny_mlp(x, weights, hidden, output, *, batch, input_dim, hidden_dim, output_dim):
```

```
    with profiler.scope("tiny_mlp"):
```

```
        with profiler.scope("1_layer1"):
```

```
            with profiler.scope("linear"):
```

```
                _linear(x, weights["w1"], weights["b1"], hidden, batch, hidden_dim, input_dim)
```



```
            with profiler.scope("2_activation"):
```

```
                with profiler.scope("relu"):
```

```
                    _relu(hidden, hidden, batch * hidden_dim)
```

```
            with profiler.scope("3_layer2"):
```

```
                with profiler.scope("linear"):
```

```
                    _linear(hidden, weights["w2"], weights["b2"], output, batch, output_dim, hidden_dim)
```

profile.timeline.json

Chrome Trace Event; Perfetto

profile.hatchet

调用树及聚合指标; proton-viewer

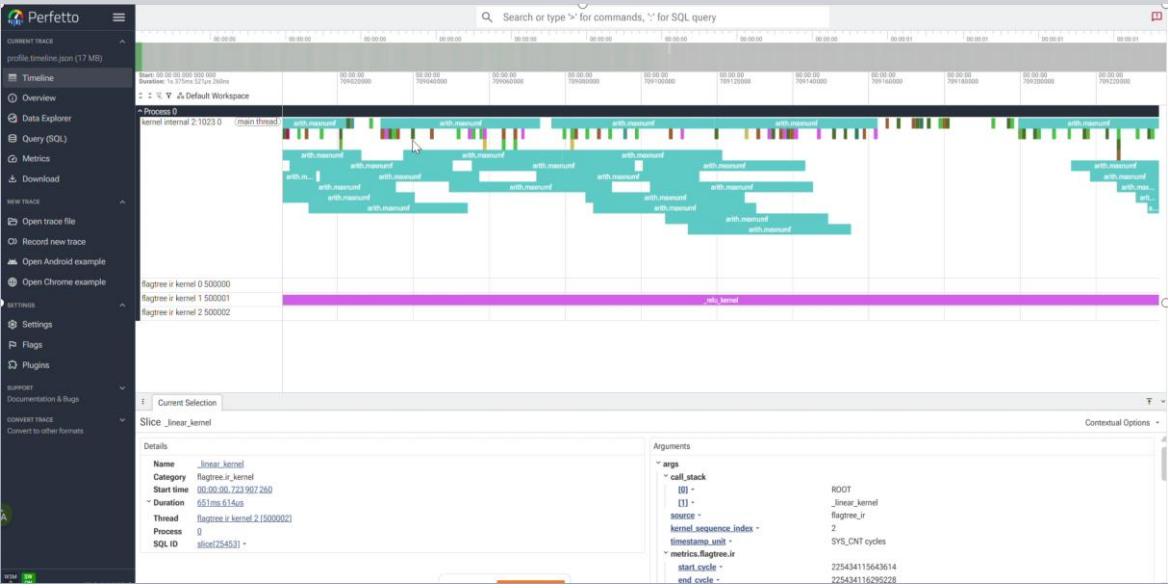
profile.meta.json

会话配置、版本、数据源和降级原因

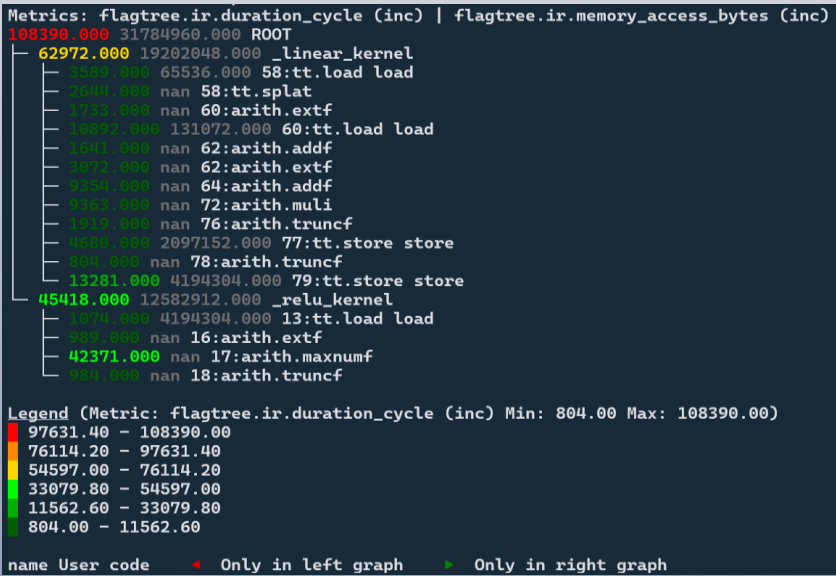
profile.vendor.json

CANN 原始字段、task 与 scope 关联

Perfetto可视化



聚合指标可视化



实机演示

FlagPrism仓库: <https://github.com/flagos-ai/FlagPrism>

时间	进度
2026.04-07	在昇腾、天数上完成 Profiler 和 Debugger 原型
2026.08	固化v0.1版本, 合入FlagTree仓库
2026.09	适配其它芯片单位
2026.10	适配其它芯片单位
2026.11	适配其它芯片单位
2026.12	发布v1.0版本, 包含不少于6种后端芯片

谢 谢

众智FlagOS：面向多元 AI 硬件统一系统软件平台

更多活动和会议请关注官网：flagos.io



FlagOS
公众号



FlagOS
仓地址



FlagOS
技术交流群